

GREYNOC

CYBERSECURITY OPERATIONS · MICHIGAN

PUBLIC

OPERATOR FIELD MANUAL

The White-Hat Scripting Manual

From First Line to Defensive Automation at Scale



```
$ greynoc init --profile whitehat --authz signed  
[ok] scope verified · cleanup armed · logging on
```

DOCUMENT ID
GN-OPS-SCRIPT-001

VERSION
v2.2

CLASSIFICATION
PUBLIC

PUBLISHED
2026-06-14

Document Control

Field	Value
Document ID	GN-OPS-SCRIPT-001
Title	The White-Hat Scripting Manual — From First Line to Defensive Automation at Scale
Version	v2.2
Classification	PUBLIC — distribution permitted with attribution
Published	2026-06-14
Owner	GreyNOC Operations
Template	GN-TPL-PUB-001
Scope	Bash · PowerShell · Python scripting for IT and white-hat / defensive security
Audience	Operators from beginner to advanced; pentesters, blue teams, sysadmins

Revision History

Version	Date	Summary
v1.0	2026-06-14	Initial public release: fundamentals plus a short defensive playbook.
v2.0	2026-06-14	Major expansion into a full manual: simple→advanced ladder, patterns & models, 14 worked defensive scripts, operational privacy & cleanup, references, and a pull-out quick-reference card.
v2.1	2026-06-14	Readability pass: brightened example/code text to near-white on a darker code background for higher contrast.
v2.2	2026-06-14	Readability pass: set example/code text in bold for stronger legibility on the dark background.

How This Manual Is Organized

The manual climbs deliberately. Each part assumes the one before it, and every chapter is tagged with a difficulty level so you can enter at the right rung and progress.

Part	Level	What you get
A — Foundations	Foundational	What scripts are, the languages, anatomy, your first working scripts
B — Engineering	Intermediate	Analysis, building discipline, safe execution, validation, error handling
C — Advanced Craft	Advanced	Language mastery, patterns & models, secrets, logging, supply chain
D — Defensive Playbook	Advanced	14 worked blue-team scripts mapped to NIST CSF 2.0
E — Privacy & Cleanup	Expert	Artifact lifecycle, secure deletion, self-cleaning scripts, engagement hygiene
F — Reference	Reference	Quick-reference card, templates, checklists, glossary, sources

Legal & Ethical Use

White-hat means lawful and authorized. Every technique in this manual is intended for systems you own or have explicit, written permission to test. Automation amplifies both help and harm: a script run at scale against the wrong target, or without authorization, can cause real damage and real legal consequences.

RULES OF ENGAGEMENT. Operate only within a signed scope. Confirm written authorization and rules of engagement before running anything against a system you do not own. Stay in scope, do no harm, handle collected data as confidential, and document what you ran and what you cleaned up.

How to Read the Callouts

Marker	Meaning
TIP	A practice that materially improves quality, safety, or maintainability.
CAUTION	A common pitfall or a place where things quietly go wrong.
DANGER	An action that can cause irreversible damage if mishandled.
NOTE	Context or clarification worth keeping in mind.
RULES OF ENGAGEMENT	An authorization, scope, or data-handling obligation for white-hat work.

Contents

Revision History	2
How This Manual Is Organized	2
Legal & Ethical Use	3
How to Read the Callouts	3
Part A • Foundations	6
1. What Is a Script?	6
2. A Brief History	6
3. Choosing a Language	6
4. The Anatomy of a Script	7
5. Your First Useful Scripts	8
Part B • Engineering Scripts	9
6. Technical Analysis — Ten Lenses	9
7. Building Scripts to a Standard	10
8. Running Scripts Safely	11
Part C • Advanced Craft	12
9. Bash Mastery	12
10. Python Production Patterns	12
11. PowerShell Production Patterns	13
12. Patterns & Models	14
13. Secrets Management	15
14. Logging, Telemetry & SIEM-Friendly Output	15
15. Supply Chain & CI Hygiene	16
Part D • Defensive Scripting Playbook	16
16. The Defensive Automation Mindset	16
17. Idea Catalog	16
18. Worked Examples	17
Part E • Operational Privacy & Cleanup	25
19. The Artifact Lifecycle Model	25
20. Secure Handling While Running	26
21. Cleanup Patterns	26

22. Secure Deletion & Data Sanitization	27
23. Scrubbing Secrets From History, Logs & Environment	27
24. Post-Engagement Hygiene Checklist	28
25. Self-Cleaning Script Templates	28
Part F · Reference	29
26. Quick-Reference Card	30
27. Standard Templates	31
28. Operator Checklists	31
29. Glossary	32
30. References & Further Reading	32
Closing	33

Part A · Foundations

● LEVEL — FOUNDATIONAL

Start here if scripting is new. By the end of this part you will understand what a script is, how to choose a language, the building blocks every script shares, and how to write small scripts that actually do useful work.

1. What Is a Script?

A script is a set of instructions, written in a scripting language, that tells a computer to perform specific tasks. Compared with full applications, scripts are smaller and more focused — designed to automate work or to control existing programs, services, and operating-system functions. A script is a technical recipe: a clear, ordered sequence the machine follows.

Wherever a task is repetitive, error-prone by hand, or needs to run unattended — collecting system information, parsing logs, scanning a network you are authorized to test, backing up data, testing controls, generating reports — a script earns its place.

TIP. If you can write the task down as a clear sequence of steps, you can usually script it. If you cannot describe it clearly, that is the signal to define the goal before writing any code.

2. A Brief History

Scripting began as a way to automate command-line work. Unix popularized small, composable tools and the shells (sh, bash, zsh) that glue them together. Windows moved from batch files to PowerShell, which works with structured objects rather than plain text. Python rose to dominate cross-platform automation, security tooling, and data work. Modern scripting is less about saving keystrokes and more about building repeatable, controlled, auditable processes.

3. Choosing a Language

Language	Strong fit for	Typical use
Bash	Linux / Unix, servers, DevOps	File ops, service control, cron, glue between CLI tools
PowerShell	Windows, Microsoft ecosystem	AD, M365, Azure, endpoint and security reporting
Python	Cross-platform, advanced automation	APIs, parsing, security tooling, reusable utilities
JavaScript	Web, Node.js	Browser behavior, APIs, server-side automation
Go / Rust	Compiled tooling	Fast, portable single-binary security tools (beyond scripting)

TIP. Cross-platform task? Reach for Python. Deep inside one OS? The native shell is usually simpler and faster. The best language is the one that matches the system you are operating.

4. The Anatomy of a Script

Syntax differs between languages, but the structural elements are remarkably consistent: comments, variables, input, output, conditionals, loops, functions, error handling, and exit codes.

4.1 Comments — explain why, not what

```
# Weak: restates the obvious
print(name)

# Strong: explains intent
# Display the current user being processed in the audit report
print(name)
```

4.2 Variables, input, output

```
name = input("Enter your name: ") # input from the user
greeting = f"Hello, {name}"       # store in a variable
print(greeting)                  # output to the screen
```

4.3 Conditionals and loops

```
for host in ["web01", "web02", "db01"]:
    if host.startswith("db"):
        print(f"{host}: database tier")
    else:
        print(f"{host}: web tier")
```

4.4 Functions and error handling

```
def read_file(path):
    try:
        with open(path) as f:
            return f.read()
    except FileNotFoundError:
        print(f"Missing: {path}")
        return None
```

4.5 Exit codes — the contract with automation

Exit codes tell schedulers, pipelines, and monitoring whether a run succeeded. By convention 0 is success and non-zero is failure.

Code	Conventional meaning
0	Success
1	General / catch-all error
2	Misuse — bad arguments or usage
3–125	Application-specific failures you define
126	Command found but not executable
127	Command not found
130	Terminated by Ctrl+C (SIGINT)

5. Your First Useful Scripts

Begin tiny, then grow. Below, a one-line idea becomes a small, dependable tool across three drafts — the same progression you will repeat for every script you write.

Draft 1 — make it work

```
import shutil
total, used, free = shutil.disk_usage("/")
print(used / total * 100)
```

Draft 2 — make it readable and decisive

```
import shutil
THRESHOLD = 80
total, used, _ = shutil.disk_usage("/")
pct = used / total * 100
print(f"WARNING: {pct:.1f}%" if pct >= THRESHOLD else f"OK: {pct:.1f}%")
```

Draft 3 — make it safe and automatable

```
"""disk_check.py – warn when disk usage exceeds a threshold."""
import shutil, sys

THRESHOLD = 80

def main() -> int:
    total, used, _ = shutil.disk_usage("/")
    pct = used / total * 100
    if pct >= THRESHOLD:
        print(f"WARNING: disk usage {pct:.1f}%")
        return 1
    print(f"OK: disk usage {pct:.1f}%")
    return 0

if __name__ == "__main__":
    sys.exit(main())
```


TIP. Every good script grows through these three drafts: make it work, make it clear, make it safe and automatable. Skipping straight to clever is how fragile scripts are born.

Part B · Engineering Scripts

● LEVEL — INTERMEDIATE

Foundations let you write scripts. This part is about writing scripts you can *trust* — analyzed before they run, built to a standard, and executed safely.

6. Technical Analysis — Ten Lenses

Before you trust a script — yours, inherited, or downloaded — run it through these lenses.

Lens	The question it answers
Purpose	What exactly is this supposed to do?
Environment	What OS, interpreter, and permissions does it assume?
Dependencies	What modules, tools, keys, or network access does it need?
Input	Where does input come from, and is it trusted and validated?
Logic	Does it make correct decisions, including on edge cases?
Output	What does it print, write, or overwrite — and does it leak data?
Security	Does it need privilege, handle secrets, or run remote code?
Performance	Will it scale to the real data volume without choking?
Reliability	Does it handle failure and run safely more than once?
Maintainability	Can the next person understand and change it?

6.1 The injection lens, in detail

The single most important input question for white-hat work: can untrusted input change which command runs? Never build a shell command by gluing strings together.

```
# UNSAFE – invites command injection
import os
target = input("host: ")
os.system("ping " + target)          # "8.8.8.8; rm -rf ~" would execute

# SAFER – arguments as a list; no shell string parsing
import subprocess
target = input("host: ")
subprocess.run(["ping", "-c", "4", target], check=True)
```

DANGER. Never concatenate untrusted input into a command string, and avoid `shell=True`. Pass arguments as a list. This one habit eliminates an entire class of vulnerabilities (OWASP: OS Command Injection).

7. Building Scripts to a Standard

1. **Define the goal** — what it does, the system it touches, inputs, outputs, and what success looks like.
2. **Choose the language** to match environment and task.
3. **Plan** the steps in plain language first.
4. **Build a minimum working version** of the core task.
5. **Add logic, then error handling, then documentation.**
6. **Test** under normal and abnormal conditions, then **review** before production.

7.1 Start with a header

```
"""
GreyNOC Script
Name: fim.py   Purpose: detect changes to watched files.
Author: GreyNOC   Version: 1.0   Date: 2026-06-14
Requirements: Python 3   Usage: python fim.py /etc --init
"""
```

7.2 Name clearly; separate configuration

```
# Configuration sits at the top, easy to change and review
AUTH_LOG_PATH      = "/var/log/auth.log"
FAILED_LOGIN_LIMIT = 10
REPORT_FILE        = "failed_login_report.txt"
```

7.3 Validate input and log key events

```
import logging
logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger("audit")

value = input("port: ")
if not value.isdigit() or not (1 <= int(value) <= 65535):
    log.warning("Rejected invalid port: %r", value)
else:
    log.info("Accepted port %s", value)
```

7.4 Never hardcode secrets

```
import os
api_key = os.getenv("GN_API_KEY")      # from the environment, never the source
if not api_key:
    raise SystemExit("Set GN_API_KEY before running.")
```

CAUTION. A secret committed to version control is compromised even after deletion — it remains in history. Rotate any credential that has ever been committed, and keep secrets out of source from the very first line.

7.5 Use version control

```
git add fim.py
git commit -m "Add file integrity monitor with JSON baseline"
```

8. Running Scripts Safely

Language	Run	Notes
Python	<code>python3 script.py</code>	Prefer python3; pin the interpreter in cron
Bash	<code>./script.sh</code>	Needs <code>chmod +x</code> and a <code>#!/bin/bash</code> shebang
PowerShell	<code>.\script.ps1</code>	Execution policy may block; scope changes narrowly

8.1 Scheduling and arguments

```
# cron — run a verified backup daily at 02:00, log output
0 2 * * * /usr/bin/python3 /opt/greynoc/backup.py >> /var/log/gn_backup.log 2>&1
```

8.2 The safe-execution checklist

- Read the entire script and confirm its source before running it.
- Identify anything that deletes, overwrites, formats, or disables.
- Run with least privilege; test in a lab or with a dry run first.
- Back up important data; watch output live; review logs afterward.

DANGER. Never pipe a remote script straight into a shell — `curl http://site/x.sh | bash` runs unseen code instantly. Download, read it, verify the source, then run it.

8.3 High-risk commands to review carefully

Command	Why it is dangerous
<code>rm -rf <path></code>	Recursive, forced deletion — no undo
<code>Remove-Item -Recurse -Force</code>	PowerShell bulk irreversible deletion
<code>dd / mkfs / blkdiscard</code>	Can overwrite or wipe entire disks
<code>chmod -R / chown -R /</code>	Sweeping permission changes can break a system
<code>:(){ : :& };;:</code>	A fork bomb — exhausts system resources

Part C · Advanced Craft

● LEVEL — ADVANCED

These are the habits and structures that make a script trustworthy unattended at scale. Most cost only a few lines and pay for themselves the first time something goes wrong at 3 a.m.

9. Bash Mastery

Begin every serious Bash script with strict mode and a cleanup trap. Quote everything.

```
#!/bin/bash
set -euo pipefail          # exit on error / unset var / failed pipe
IFS=$'\n\t'                # safer word-splitting

WORKDIR="$(mktemp -d)"      # private temp directory
cleanup() { rm -rf "$WORKDIR"; }
trap cleanup EXIT INT TERM  # always runs, even on error or Ctrl+C

usage() { echo "usage: $0 -t <target> [-v]" >&2; exit 2; }
verbose=0; target=""
while getopts ":t:v" opt; do
    case "$opt" in
        t) target="$OPTARG" ;;
        v) verbose=1 ;;
        *) usage ;;
    esac
done
[[ -n "$target" ]] || usage
echo "Target: $target (workdir: $WORKDIR)"
```

- **set -e / -u / -o pipefail** turn silent failures into loud, early ones.
- **trap cleanup EXIT INT TERM** guarantees temp data is removed however the script ends.
- **getopts** gives proper flag parsing instead of brittle positional arguments.
- Quote every variable ("\$var"); prefer `[[ ]]` and arrays over word-split strings.

TIP. Run every shell script through **ShellCheck** before committing. It catches quoting bugs, unsafe patterns, and portability issues the eye misses.

10. Python Production Patterns

```
#!/usr/bin/env python3
"""Production-shaped CLI skeleton."""
import argparse, logging, sys
from pathlib import Path

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(name)s %(message)s")
log = logging.getLogger("tool")

def parse_args() -> argparse.Namespace:
    p = argparse.ArgumentParser(description="What this tool does.")
    p.add_argument("path", type=Path)
    p.add_argument("--threshold", type=int, default=80)
    p.add_argument("--dry-run", action="store_true")
    return p.parse_args()

def main() -> int:
    args = parse_args()
    log.info("Starting on %s (dry-run=%s)", args.path, args.dry_run)
    # ... work ...
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

- **argparse** over hand-parsing `sys.argv` — free help text, types, validation.
- **logging** over `print` — levels, timestamps, routing to files or SIEM.
- **pathlib.Path** for cross-platform paths; **type hints** for clarity and tooling.
- A **--dry-run** flag so the tool can show intent without acting.
- Pin dependencies in `requirements.txt`; work inside a virtual environment.

10.1 Resilient external calls — retry with backoff

```
import time

def with_retry(fn, attempts=3, base=0.5):
    """Retry fn with exponential backoff; re-raise on final failure."""
    for i in range(attempts):
        try:
            return fn()
        except Exception as e:
            if i == attempts - 1:
                raise
            wait = base * (2 ** i)
            time.sleep(wait)
```

11. PowerShell Production Patterns

```
<#
.SYNOPSIS Restart a service safely, with -WhatIf support.
#>
[CmdletBinding(SupportsShouldProcess)]
param(
    [Parameter(Mandatory)][ValidateNotNullOrEmpty()]
    [string]$ServiceName
)
Set-StrictMode -Version Latest
$ErrorActionPreference = "Stop"

try {
    if ($PSCmdlet.ShouldProcess($ServiceName, "Restart service")) {
        Restart-Service -Name $ServiceName
        Write-Output "Restarted $ServiceName"
    }
}
catch {
    Write-Error "Failed: $_"
    exit 1
}
```

- **SupportsShouldProcess** gives -WhatIf and -Confirm automatically.
- **Set-StrictMode** + **\$ErrorActionPreference='Stop'** surface silent failures.
- Validate parameters with attributes such as **[ValidateNotNullOrEmpty()]**.
- Use approved verbs and comment-based help; avoid **Invoke-Expression** on untrusted input.

12. Patterns & Models

Most robust scripts are instances of a few reusable models. Recognizing them turns a blank page into an assembly job.

Model	Idea	Where it shows up
Baseline & diff	Snapshot a known-good state; alert on deviation	FIM, config drift, port and account auditing
Observer / monitor	Watch a source, emit events on conditions	Log triage, health checks, brute-force detection
Pipeline / filter	Pass data through composable stages	Log parsing, enrichment, reporting
Idempotent converge	Describe desired state; re-run is safe	Provisioning, hardening, cleanup
Retry with backoff	Tolerate transient failure gracefully	API and network calls
Fail-safe / fail-secure	Choose the safe default when uncertain	Access decisions, error paths
Least privilege	Grant only what the task needs	Every script that touches a real system
Defense in depth	Layer independent safeguards	Validation + sandboxing + monitoring together

NOTE. Fail-safe vs fail-secure: a fail-safe lock opens on power loss (safety first); a fail-secure lock stays locked (security first). Decide deliberately which your script is when an error interrupts it mid-run — the default behavior is a security decision.

13. Secrets Management

Credentials are the highest-value thing a script touches. Keep them out of source, out of arguments, and out of logs.

Approach	Rating	Notes
Managed secret store / vault	Best	Central rotation, access control, audit (e.g., HashiCorp Vault, cloud KMS)
Managed / workload identity	Best	No long-lived secret on disk at all
OS keyring / credential manager	Good	Per-user encrypted storage
Environment variable	Acceptable	Convenient; beware process listings and child inheritance
Restricted config file (chmod 600)	Acceptable	Outside the repo; never world-readable
Hardcoded in source	Never	Compromised the moment it is committed

CAUTION. Secrets passed as command-line arguments are visible to other users via the process list (ps) and often land in shell history. Prefer environment variables, a file descriptor, or stdin.

14. Logging, Telemetry & SIEM-Friendly Output

Logs are how unattended scripts earn trust. Make them structured, timestamped, and free of secrets so they feed straight into a SIEM.

```
import json, logging, sys

class JsonFormatter(logging.Formatter):
    def format(self, r):
        return json.dumps({
            "ts": self.formatTime(r), "level": r.levelname,
            "logger": r.name, "msg": r.getMessage(),
        })

h = logging.StreamHandler(sys.stdout)
h.setFormatter(JsonFormatter())
log = logging.getLogger("gn"); log.addHandler(h); log.setLevel(logging.INFO)
log.info("scan complete")      # -> {"ts": "...", "level": "INFO", ...}
```

DANGER. Never log secrets, tokens, full request bodies, or personal data. A log file is just another place a credential can leak — redact before you write.

15. Supply Chain & CI Hygiene

- **Pin dependencies** to exact versions and verify checksums; review what you install.
- **Lint and test in CI** — ShellCheck, ruff/flake8, PSScriptAnalyzer, plus unit tests on logic.
- **Sign or hash** released scripts so consumers can verify integrity before running.
- **Least-privilege CI** — scoped tokens, no long-lived secrets in the pipeline.

Part D · Defensive Scripting Playbook

● LEVEL — ADVANCED

Automation is a force multiplier for defenders. Every script below is **read-and-alert** by design — it observes, baselines, and reports rather than taking destructive action — which makes it safe to deploy early and run on a schedule. Adapt paths, thresholds, and alerting to your environment, and test in a lab first.

16. The Defensive Automation Mindset

- **Baseline, then detect drift.** Most detection is comparing now against a known-good snapshot.
- **Prefer observation over action.** Alert a human before automating anything destructive.
- **Make output SIEM-friendly.** Structured, timestamped lines integrate with existing tooling.
- **Fail safe and loud.** A monitor that dies silently is worse than no monitor at all.

The catalog below maps each script to a function of the **NIST Cybersecurity Framework 2.0** (Govern, Identify, Protect, Detect, Respond, Recover) so the automation ties back to a recognized control structure.

17. Idea Catalog

Script	CSF function	What it does
Failed-login monitor	Detect	Counts auth failures per source IP; flags brute force
File integrity monitor	Detect	Hashes critical files; reports any change/add/remove
TLS expiry checker	Protect	Warns before certificates expire and cause outages
Listening-port baseline	Detect	Differs open ports against an approved allowlist
Verified backup	Recover	Creates backups and proves integrity with checksums
Resource health monitor	Detect	Tracks disk, memory, CPU against thresholds
IOC enrichment	Respond	Looks indicators up against local intel / pluggable feeds
Secret scanner	Protect	Finds keys and tokens accidentally committed to a repo
Config drift detector	Protect	Compares live config to an approved baseline
New-account auditor	Detect	Flags newly created or newly privileged accounts
SUID / SGID auditor	Identify	Inventories privilege-granting binaries vs allowlist
Cron / task auditor	Detect	Baselines scheduled jobs; reports unexpected changes
Web availability probe	Detect	Checks reachability, status, latency, and cert health
Log triage summarizer	Respond	Buckets log levels and flags error spikes

18. Worked Examples

18.1 Failed-Login Monitor — Bash

```
#!/bin/bash
# GreyNOC - failed_login_monitor.sh : report IPs over a failed-login threshold.
set -euo pipefail
AUTH_LOG="${1:-/var/log/auth.log}"
THRESHOLD="${2:-10}"
[[ -r "$AUTH_LOG" ]] || { echo "ERROR: cannot read $AUTH_LOG" >&2; exit 1; }

grep -a "Failed password" "$AUTH_LOG" \
  | grep -aoE "from [0-9]+\.[0-9]+\.[0-9]+\.[0-9]+" \
  | awk '{print $2}' | sort | uniq -c | sort -rn \
  | while read -r count ip; do
    (( count >= THRESHOLD )) && echo "[ALERT] $ip - $count failed attempts"
  done
echo "[INFO] Scan complete"
```

NOTE. For automated blocking, pair this with a maintained tool such as fail2ban rather than scripting bans by hand — the edge cases (allowlists, lockout windows) are easy to get wrong.

18.2 File Integrity Monitor — Python

```
#!/usr/bin/env python3
"""GreyNOC - fim.py : baseline and detect changes to watched files."""
import argparse, hashlib, json, sys
from pathlib import Path

def sha256(p: Path) -> str:
    h = hashlib.sha256()
    with p.open("rb") as f:
        for chunk in iter(lambda: f.read(8192), b''):
            h.update(chunk)
    return h.hexdigest()

def snapshot(paths):
    return {str(p): sha256(p)
            for base in paths for p in Path(base).rglob("*") if p.is_file()}

def main() -> int:
    ap = argparse.ArgumentParser(description="File integrity monitor.")
    ap.add_argument("paths", nargs="+")
    ap.add_argument("--baseline", default="fim_baseline.json")
    ap.add_argument("--init", action="store_true")
    args = ap.parse_args()
    cur, bf = snapshot(args.paths), Path(args.baseline)
    if args.init or not bf.exists():
        bf.write_text(json.dumps(cur, indent=2))
        print(f"[INFO] Baseline written: {bf} ({len(cur)} files)"); return 0
    old = json.loads(bf.read_text())
    for p in sorted(cur.keys() - old.keys()): print(f"[ALERT] ADDED {p}")
    for p in sorted(old.keys() - cur.keys()): print(f"[ALERT] REMOVED {p}")
    for p in sorted(k for k in cur.keys() & old.keys() if cur[k] != old[k]):
        print(f"[ALERT] CHANGED {p}")
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

18.3 TLS Certificate Expiry Checker — Python

```
#!/usr/bin/env python3
"""GreyNOC – cert_expiry.py : warn before a TLS certificate expires."""
import argparse, socket, ssl, sys
from datetime import datetime, timezone

def days_left(host: str, port: int = 443) -> int:
    ctx = ssl.create_default_context()
    with socket.create_connection((host, port), timeout=8) as s:
        with ctx.wrap_socket(s, server_hostname=host) as ss:
            cert = ss.getpeercert()
            exp = datetime.strptime(cert["notAfter"], "%b %d %H:%M:%S %Y %Z")
            return (exp.replace(tzinfo=timezone.utc) - datetime.now(timezone.utc)).days

def main() -> int:
    ap = argparse.ArgumentParser(description="TLS expiry checker.")
    ap.add_argument("host"); ap.add_argument("--warn", type=int, default=21)
    a = ap.parse_args()
    try:
        d = days_left(a.host)
    except Exception as e:
        print(f"[ERROR] {a.host}: {e}"); return 2
    if d < 0:          print(f"[ALERT] {a.host}: certificate EXPIRED"); return 1
    if d <= a.warn:    print(f"[WARN] {a.host}: expires in {d} days"); return 1
    print(f"[OK] {a.host}: {d} days remaining"); return 0

if __name__ == "__main__":
    sys.exit(main())
```

18.4 Listening-Port Baseline & Drift — Bash

```
#!/bin/bash
# GreyNOC – port_drift.sh : flag listening ports not on the allowlist.
set -euo pipefail
ALLOWLIST="${1:-allowed_ports.txt}" # one port per line
[[ -r "$ALLOWLIST" ]] || { echo "ERROR: missing $ALLOWLIST" >&2; exit 1; }

ss -ltn | awk 'NR>1 {n=split($4,a,"."); print a[n]}' | sort -un \
| while read -r port; do
    [[ -z "$port" ]] && continue
    grep -qx "$port" "$ALLOWLIST" || echo "[ALERT] Unexpected port: $port"
done
echo "[INFO] Port audit complete"
```

18.5 Verified Backup — Bash

```
#!/bin/bash
# GreyNOC - verified_backup.sh : archive, checksum, and verify.
set -euo pipefail
SRC="${1:?usage: verified_backup.sh <source-dir> <dest-dir>}"
DEST="${2:?usage: verified_backup.sh <source-dir> <dest-dir>}"
stamp="$(date +%Y-%m-%d_%H%M%S)"; archive="$DEST/backup_${stamp}.tar.gz"
mkdir -p "$DEST"
tar -czf "$archive" -C "$(dirname "$SRC")" "$(basename "$SRC")"
sha256sum "$archive" > "$archive.sha256"
sha256sum -c "$archive.sha256"
echo "[INFO] Verified: $archive ($(du -h "$archive" | cut -f1))"
```

TIP. A backup you have never restored is a hope, not a backup. Periodically test a restore into a scratch location — the checksum step proves integrity, not recoverability.

18.6 Resource Health Monitor — Python

```
#!/usr/bin/env python3
"""GreyNOC - health_monitor.py : disk + memory check with exit codes."""
import shutil, sys
from pathlib import Path
DISK_WARN, MEM_WARN = 85, 90

def disk_pct(path="/"):
    t, u, _ = shutil.disk_usage(path); return u / t * 100

def mem_pct():
    info = {}
    for line in Path("/proc/meminfo").read_text().splitlines():
        k, v, *_ = line.replace(":", "").split(); info[k] = int(v)
    return (info["MemTotal"] - info["MemAvailable"]) / info["MemTotal"] * 100

def main() -> int:
    rc = 0
    d = disk_pct(); print(f"[{'WARN' if d>=DISK_WARN else 'OK'}] Disk {d:.1f}%")
    rc |= d >= DISK_WARN
    try:
        m = mem_pct(); print(f"[{'WARN' if m>=MEM_WARN else 'OK'}] Mem {m:.1f}%")
        rc |= m >= MEM_WARN
    except FileNotFoundError:
        print("[INFO] Memory check skipped (no /proc/meminfo)")
    return int(rc)

if __name__ == "__main__":
    sys.exit(main())
```

18.7 IOC Enrichment — Python

Reads indicators (IPs or hashes), checks them against a local intelligence list, and exposes a pluggable hook for your own threat-intel feed. No paid API is assumed — wire in whatever your program licenses.

```
#!/usr/bin/env python3
"""GreyNOC - ioc_enrich.py : match indicators against local intel."""
import argparse, json, sys
from pathlib import Path

def load_intel(path: Path) -> dict:
    # JSON: {"1.2.3.4": "known-scanner", "<sha256>": "malware-x"}
    return json.loads(path.read_text()) if path.exists() else {}

def feed_lookup(indicator: str) -> str | None:
    # Plug your licensed feed here; return a label or None.
    return None

def main() -> int:
    ap = argparse.ArgumentParser(description="IOC enrichment.")
    ap.add_argument("indicators", nargs="+")
    ap.add_argument("--intel", type=Path, default=Path("intel.json"))
    a = ap.parse_args()
    intel, hits = load_intel(a.intel), 0
    for ioc in a.indicators:
        label = intel.get(ioc) or feed_lookup(ioc)
        if label:
            hits += 1
            print(json.dumps({"indicator": ioc, "match": label}))
        else:
            print(json.dumps({"indicator": ioc, "match": None}))
    return 1 if hits else 0

if __name__ == "__main__":
    sys.exit(main())
```

18.8 Secret Scanner — Python

Scans a directory tree for credentials accidentally committed to source — a defensive scan you run against your *own* repositories to catch leaks before they ship.

```
#!/usr/bin/env python3
"""GreyNOC – secret_scan.py : find committed secrets in a tree."""
import argparse, re, sys
from pathlib import Path

PATTERNS = {
    "AWS Access Key": re.compile(r"AKIA[0-9A-Z]{16}"),
    "Private Key": re.compile(r"-----BEGIN (?:(RSA |EC |OPENSSH )?PRIVATE KEY-----"),
    "Generic API key": re.compile(r"(?i)(api[_-]?key|secret)\s*[:=]\s*['\"]?[0-9a-zA-Z]{16,}"),
    "Bearer token": re.compile(r"(?i)bearer\s+[0-9a-zA-Z._-]{20,}"),
}
SKIP = {".git", "node_modules", "__pycache__", "venv"}

def main() -> int:
    ap = argparse.ArgumentParser(description="Secret scanner.")
    ap.add_argument("root", type=Path)
    a = ap.parse_args()
    found = 0
    for p in a.root.rglob("**"):
        if not p.is_file() or any(s in p.parts for s in SKIP):
            continue
        try:
            text = p.read_text(errors="ignore")
        except Exception:
            continue
        for line_no, line in enumerate(text.splitlines(), 1):
            for name, rx in PATTERNS.items():
                if rx.search(line):
                    found += 1
                    print(f"[ALERT] {name}: {p}:{line_no}")
    print(f"[INFO] Scan complete ({found} potential secrets)")
    return 1 if found else 0

if __name__ == "__main__":
    sys.exit(main())
```

CAUTION. A secret-scanner hit means rotate first, scrub second. Removing the line does not un-leak a credential that already reached a shared repo or its history — rotate the secret, then clean the history.

18.9 Config Drift Detector — Python

```
#!/usr/bin/env python3
"""GreyNOC – config_drift.py : compare configs to an approved baseline."""
import argparse, hashlib, json, sys
from pathlib import Path

def digest(p: Path) -> str:
    return hashlib.sha256(p.read_bytes()).hexdigest()

def main() -> int:
    ap = argparse.ArgumentParser(description="Config drift detector.")
    ap.add_argument("files", nargs="+")
    ap.add_argument("--baseline", default="config_baseline.json")
    ap.add_argument("--init", action="store_true")
    a = ap.parse_args()
    cur = {f: digest(Path(f)) for f in a.files if Path(f).is_file()}
    bf = Path(a.baseline)
    if a.init or not bf.exists():
        bf.write_text(json.dumps(cur, indent=2))
        print(f"[INFO] Baseline written ({len(cur)} files)"); return 0
    old = json.loads(bf.read_text()); drift = 0
    for f, h in cur.items():
        if old.get(f) != h:
            drift += 1; print(f"[ALERT] DRIFT {f}")
    print("[OK] No drift" if not drift else f"[INFO] {drift} drifted file(s)")
    return 1 if drift else 0

if __name__ == "__main__":
    sys.exit(main())
```

18.10 New-Account & Privilege Auditor — Bash

```
#!/bin/bash
# GreyNOC – account_audit.sh : baseline users + UID-0 accounts; report drift.
set -euo pipefail
BASE="${1:-account_baseline.txt}"
cur="$(getent passwd | awk -F: '{print $1":"$3}' | sort)"
if [[ ! -f "$BASE" ]]; then
    echo "$cur" > "$BASE"; echo "[INFO] Baseline written"; exit 0
fi
echo "[INFO] UID 0 accounts:"; awk -F: '$3==0 {print "  "$1}' /etc/passwd
diff <(echo "$cur") "$BASE" | grep '^<' | sed 's/^< /[ALERT] NEW USER: /' || \
    echo "[OK] No new accounts"
```

18.11 SUID / SGID Auditor — Bash

```
#!/bin/bash
# GreyNOC – suid_audit.sh : inventory SUID/SGID binaries vs an allowlist.
set -euo pipefail
ALLOW="${1:-suid_allowlist.txt}"
touch "$ALLOW"
find / -xdev -type f \( -perm -4000 -o -perm -2000 \) 2>/dev/null | sort \
| while read -r bin; do
    grep -qx "$bin" "$ALLOW" || echo "[ALERT] Unexpected SUID/SGID: $bin"
done
echo "[INFO] SUID/SGID audit complete"
```

18.12 Cron / Scheduled-Task Auditor — Bash

```
#!/bin/bash
# GreyNOC - cron_audit.sh : baseline cron sources and report changes.
set -euo pipefail
BASE="${1:-cron_baseline.txt}"
collect() {
    cat /etc/crontab 2>/dev/null
    find /etc/cron.d /etc/cron.daily /etc/cron.hourly -type f 2>/dev/null
    for u in $(cut -f1 -d: /etc/passwd); do
        crontab -l -u "$u" 2>/dev/null | sed "s/^/[${u}] /"
    done
}
cur="$(collect | sort)"
if [[ ! -f "$BASE" ]]; then
    echo "$cur" > "$BASE"; echo "[INFO] Baseline written"; exit 0
fi
diff <(echo "$cur") "$BASE" | grep '^<' | sed 's/^< /[ALERT] NEW CRON: /' || \
    echo "[OK] No cron changes"
```

18.13 Web Availability & Cert Probe — Python

```
#!/usr/bin/env python3
"""GreyNOC - web_probe.py : reachability, status, latency, cert days."""
import sys, time, ssl, socket
from urllib.request import urlopen
from urllib.parse import urlparse
from datetime import datetime, timezone

def cert_days(host, port=443):
    ctx = ssl.create_default_context()
    with socket.create_connection((host, port), timeout=8) as s:
        with ctx.wrap_socket(s, server_hostname=host) as ss:
            exp = datetime.strptime(ss.getpeercert()["notAfter"],
                                    "%b %d %H:%M:%S %Y %Z")
    return (exp.replace(tzinfo=timezone.utc) - datetime.now(timezone.utc)).days

def probe(url):
    t0 = time.time()
    try:
        with urlopen(url, timeout=10) as r:
            ms = (time.time() - t0) * 1000
            host = urlparse(url).hostname
            days = cert_days(host) if url.startswith("https") else "n/a"
            print(f"[OK] {url} {r.status} {ms:.0f}ms cert={days}d")
            return 0
    except Exception as e:
        print(f"[ALERT] {url} unreachable: {e}")
        return 1

def main() -> int:
    return max((probe(u) for u in sys.argv[1:]), default=0)

if __name__ == "__main__":
    sys.exit(main())
```


18.14 Log Triage Summarizer — Python

```
#!/usr/bin/env python3
"""GreyNOC - log_triage.py : bucket log levels and flag error spikes."""
import argparse, re, sys
from collections import Counter
from pathlib import Path

LEVEL = re.compile(r"\b(DEBUG|INFO|WARN|WARNING|ERROR|CRITICAL)\b")

def main() -> int:
    ap = argparse.ArgumentParser(description="Log triage.")
    ap.add_argument("logfile", type=Path)
    ap.add_argument("--error-threshold", type=int, default=50)
    a = ap.parse_args()
    counts = Counter()
    for line in a.logfile.read_text(errors="ignore").splitlines():
        m = LEVEL.search(line)
        if m:
            counts[m.group(1).replace("WARNING", "WARN")] += 1
    for lvl in ("CRITICAL", "ERROR", "WARN", "INFO", "DEBUG"):
        if counts[lvl]:
            print(f" {lvl:<8} {counts[lvl]}")
    errs = counts["ERROR"] + counts["CRITICAL"]
    if errs >= a.error_threshold:
        print(f"[ALERT] Error spike: {errs} (>= {a.error_threshold})")
        return 1
    print("[OK] No error spike")
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

Part E · Operational Privacy & Cleanup

● LEVEL — EXPERT

Scripts create artifacts: temp files, logs, output, caches, and shell history. In white-hat work those artifacts often contain sensitive client data or credentials. Cleaning up afterward is part of the job — both to protect that data and to leave authorized systems as you found them, per your rules of engagement.

RULES OF ENGAGEMENT. Cleanup here means lawful, documented hygiene: protecting collected data, removing test artifacts *you* created on authorized systems with permission, and keeping your own workstation clean. It is not about evading detection or defeating an incident response — authorized engagements are documented and coordinated, not hidden.

19. The Artifact Lifecycle Model

Treat every artifact a script touches as having a lifecycle: **create** → **secure** → **use** → **sanitize** → **verify**. Designing with that lifecycle in mind is what makes cleanup reliable instead of an afterthought.

Artifact	Where it hides	Cleanup approach
Temp files	/tmp, working dirs, caches	mktemp + trap-based removal; tmpfs for sensitive data
Output / reports	Result files, exports	Store in an encrypted container; sanitize on disposal
Logs	App logs, syslog, SIEM	Redact secrets before writing; control retention
Shell history	~/.bash_history, ~/.zsh_history	Keep secrets out of argv; scrub if they slipped in
Environment	Exported variables, child procs	Unset after use; avoid passing secrets in argv
Credentials in memory	Process memory, swap	Minimize lifetime; use full-disk encryption + swap encryption

20. Secure Handling While Running

- Create sensitive temp data with a restrictive umask and 0600 permissions; prefer a private `mktemp -d` directory.
- For highly sensitive data, work in **tmpfs** (RAM-backed) so nothing durable hits the disk.
- Never place secrets in command-line arguments — they are visible in process listings.
- Set short timeouts and minimize how long credentials live in memory.

```
#!/bin/bash
set -euo pipefail
umask 077                                # new files are owner-only
WORK="$(mktemp -d)"                      # private working dir
trap 'rm -rf "$WORK"' EXIT INT TERM      # guaranteed cleanup
# For RAM-only handling of sensitive scratch data:
# mount -t tmpfs -o size=64m tmpfs "$WORK" (requires privilege)
```

21. Cleanup Patterns

Make cleanup automatic and idempotent so it runs whether the script finishes, errors, or is interrupted.

21.1 Bash — trap on EXIT

```
cleanup() {
    rm -rf "$WORK" 2>/dev/null || true
    unset API_TOKEN 2>/dev/null || true
}
trap cleanup EXIT INT TERM
```

21.2 Python — context managers and finally

```
import tempfile, os, contextlib

with tempfile.TemporaryDirectory() as work:      # auto-removed on exit
    os.chmod(work, 0o700)
    # ... use work ...
    pass                                         # cleanup is automatic

token = os.environ.pop("GN_API_TOKEN", None)     # remove from env after read
try:
    ...
finally:
    del token                                  # drop the reference
```

22. Secure Deletion & Data Sanitization

Deleting a file usually just unlinks it; the data remains until overwritten. But the classic advice to “overwrite with random data” no longer holds on modern storage, and getting this wrong gives false confidence.

CAUTION. On SSDs and flash, wear-leveling and over-provisioning mean an overwrite may never touch the original cells. On journaling and copy-on-write filesystems (ext4 journal, Btrfs, ZFS) and anywhere snapshots or backups exist, single-file overwrite tools like `shred` are unreliable.

Current guidance — **NIST SP 800-88 Rev. 2** (effective September 2025), which now defers technique selection to **IEEE 2883-2022** — takes a risk-based view and favors **cryptographic erase**: keep sensitive data only inside an encrypted volume, and destroy the key to render the data unrecoverable. For whole devices, use the drive's built-in secure-erase or sanitize command rather than software overwrite passes.

Goal	Reliable approach
Protect engagement data at rest	Full-disk or volume encryption (LUKS, BitLocker, FileVault, VeraCrypt)
Destroy a set of sensitive files	Keep them in an encrypted container; destroy the container and key
Sanitize a whole SSD/NVMe	Drive secure-erase / NVMe sanitize / ATA secure erase, then verify
Sanitize a magnetic HDD	Overwrite (e.g., <code>shred</code>) is acceptable on non-journaled HDD; verify
Highest assurance / end of life	Physical destruction per NIST 800-88 r2 ‘Destroy’

TIP. The most practical operator habit: never let sensitive client data exist in plaintext at rest. Store it inside an encrypted volume from the start, and cryptographic erase becomes as simple as destroying the key.

23. Scrubbing Secrets From History, Logs & Environment

- Prevent first: keep secrets out of `argv` and out of log statements so they never reach history or logs.
- In Bash, a leading space keeps a command out of history when `HISTCONTROL=ignorespace` is set — useful, but prevention beats relying on it.
- If a secret did land in history, remove the entry and then **rotate the secret** — the entry may already be backed up.
- Redact secrets from logs before writing; scrub existing log lines only with authorization and a record.

```
# Keep a one-off command out of bash history (HISTCONTROL=ignorespace)
export GN_API_TOKEN="..."      # note the leading space

# Remove a specific history entry, then rotate the credential
history | grep TOKEN             # find the line number N
history -d N
# ... now rotate the token at the provider; deletion alone is not enough
```

24. Post-Engagement Hygiene Checklist

- Confirm all test artifacts you placed on authorized systems are removed, and documented as removed.
- Move collected data into an encrypted container; remove plaintext working copies.
- Rotate any credentials that were generated, used, or exposed during the engagement.
- Verify cleanup with a checklist and capture it in the engagement record.
- Sanitize or encrypt the workstation storage that held client data, per your data-handling policy.

RULES OF ENGAGEMENT. Document what you did and what you removed. In white-hat work, a clean, auditable trail of your own actions protects both the client and you.

25. Self-Cleaning Script Templates

25.1 Bash

```
#!/bin/bash
# GreyNOC self-cleaning template
set -euo pipefail
umask 077
WORK="$(mktemp -d)"
cleanup() {
    rm -rf "$WORK" 2>/dev/null || true
    # unset any sensitive vars used during the run
    unset GN_API_TOKEN 2>/dev/null || true
}
trap cleanup EXIT INT TERM

echo "[INFO] Working in $WORK"
# --- sensitive work confined to $WORK ---
echo "[INFO] Done; cleanup runs automatically on exit"
```

25.2 Python

```
#!/usr/bin/env python3
"""GreyNOC self-cleaning template."""
import os, sys, tempfile

def main() -> int:
    with tempfile.TemporaryDirectory() as work: # removed on exit
        os.chmod(work, 0o700)
        token = os.environ.pop("GN_API_TOKEN", None)
        try:
            # --- sensitive work confined to `work`, token in memory only ---
            print(f"[INFO] Working in {work}")
            return 0
        finally:
            del token # drop the reference
            # TemporaryDirectory cleans `work` automatically

if __name__ == "__main__":
    sys.exit(main())
```

Part F · Reference

● LEVEL — REFERENCE

26. Quick-Reference Card

Designed as a one- to two-page pull-out. Print and keep at the console.

Strict-mode openers

```
# Bash
set -euo pipefail; IFS=$'\n\t'; umask 077
trap 'rm -rf "$WORK"' EXIT INT TERM

# Python
import argparse, logging, sys; from pathlib import Path

# PowerShell
Set-StrictMode -Version Latest; $ErrorActionPreference = "Stop"
```

Safe-by-default reflexes

Do	Instead of
<code>subprocess.run(..., check=True)</code>	<code>os.system("cmd " + userinput)</code>
Quote "\$var"; use arrays	Unquoted \$var word-splitting
<code>os.getenv("KEY")</code>	<code>key = "hardcoded-secret"</code>
<code>SupportsShouldProcess / --dry-run</code>	Acting immediately with no preview
<code>mktemp -d + trap cleanup</code>	Hand-named temp files left behind

Exit codes

Code	Meaning
0	Success
1	General error
2	Usage / bad args
126/127	Not executable / not found
130	Ctrl+C (SIGINT)

Cleanup & secrets reflexes

- Encrypt sensitive data at rest; cryptographic erase by destroying the key.
- Never put secrets in argv; redact before logging; rotate after any exposure.
- `shred` is unreliable on SSD/journaled FS — prefer encryption + key destruction.
- trap-based cleanup on EXIT INT TERM; idempotent and quiet.

Debug one-liners

```
print(f"DEBUG {x=}")          # Python (3.8+)
echo "DEBUG: x=$x" >&2         # Bash
Write-Output "DEBUG: x=$x"    # PowerShell
bash -x script.sh             # trace Bash execution
```

27. Standard Templates

27.1 Python

```
"""GreyNOC Script – Name / Purpose / Author / Version / Date / Usage"""
import sys
def main() -> int:
    try:
        # --- logic ---
        return 0
    except Exception as e:
        print(f"ERROR: {e}"); return 1
if __name__ == "__main__":
    sys.exit(main())
```

27.2 Bash

```
#!/bin/bash
# GreyNOC Script – Name / Purpose / Author / Version / Date / Usage
set -euo pipefail
trap 'rm -rf "${WORK:-}" 2>/dev/null || true' EXIT INT TERM
# --- logic ---
exit 0
```

27.3 PowerShell

```
<# GreyNOC Script – Name / Purpose / Author / Version / Date / Usage #>
[CmdletBinding(SupportsShouldProcess)] param()
Set-StrictMode -Version Latest; $ErrorActionPreference = "Stop"
try { <# logic #>; exit 0 } catch { Write-Error "ERROR: $_"; exit 1 }
```

28. Operator Checklists

Planning

- Purpose, target, and language clear.
- Inputs/outputs defined; permissions and risks understood.

Code

- Header, clear names, functions.
- Input validated; errors handled; meaningful exit codes.
- No hardcoded secrets; dangerous commands reviewed; key events logged.

Testing

- Tested in a lab with valid and invalid input.
- Missing-file and no-privilege cases tested; logs reviewed.

Security & Privacy

- Source trusted and reviewed; least privilege.
- Secrets protected and never logged.
- Cleanup armed (trap/finally); sensitive data encrypted at rest.
- Post-engagement artifacts removed and documented.

29. Glossary

Term	Meaning
Idempotent	Safe to run more than once with the same result and no extra harm
Baseline	A known-good snapshot used to detect later drift
IOC	Indicator of Compromise — an IP, hash, or pattern tied to malicious activity
Cryptographic erase	Rendering data unrecoverable by destroying the encryption key
Least privilege	Granting only the access a task strictly requires
Dry run	Executing a script in a mode that previews changes without making them
Rules of engagement	The agreed scope, authorization, and limits of an authorized test

30. References & Further Reading

- NIST Cybersecurity Framework 2.0 (2024) — Govern, Identify, Protect, Detect, Respond, Recover.
- NIST SP 800-88 Rev. 2 (2025), Guidelines for Media Sanitization; IEEE 2883-2022 for technique selection.
- OWASP — OS Command Injection and Secure Coding Practices guidance.
- ShellCheck — static analysis for shell scripts; Google Shell Style Guide.
- Python documentation — logging, argparse, subprocess, secrets.
- Microsoft Learn — PowerShell approved verbs and ShouldProcess.
- MITRE ATT&CK; and D3FEND — adversary techniques and defensive countermeasure taxonomies.
- CIS Benchmarks — configuration baselines for drift detection.

NOTE. Standards and tools evolve. Confirm the current revision of any referenced publication before relying on it for a control or an audit.

Closing

A good script is not merely code that works. It is understandable, safe, tested, documented, repeatable, and built for the environment where it will run — and, in white-hat work, it cleans up after itself and respects the scope it was authorized within. Climb the ladder, reuse the models, and let automation make your operations both stronger and safer.

TIP. Build scripts that are useful, safe, repeatable, and trusted. Everything in this manual serves that one goal.

GreyNOC · greynoc.com · Certified Penetration Testing & Ethical Hacking · Michigan

GN-OPS-SCRIPT-001 · v2.2 · Classification: PUBLIC · Published 2026-06-14

For lawful, authorized, defensive and educational use only. Adapt and test all examples in a controlled environment, and operate within a signed scope.